CSC311: Introduction to Machine Learning

Project Proposal

Submitted by

Group Number: 47982

Alexandros Pratsos, Amaan Baweja, Kabir Kumar, Adam Petrovic

University of Toronto
Winter 2026

1 Executive Summary

We will be classifying student survey responses about three paintings *The Persistence of Memory*, *The Starry Night*, and *The Water Lily Pond* using three model families: Logistic Regression, Random Forest, and a feed-forward Neural Network. Our data exploration reveals that the four Likert emotion scores (sombre, content, calm, uneasy) provide strong class separation, while the TF-IDF representations of text responses and multi-hot encoded categorical features will offer complementary discriminative signal. We plan to tune these models across their respective hyperparameter spaces using a grouped 60/20/20 train-validation-test split that prevents student-level data leakage, and evaluate performance using accuracy, precision, recall, and F1 score.

2 Data Exploration

2.1 Dataset Summary

The dataset contains survey responses from 562 students, each evaluating three paintings, giving us 1,686 rows and 16 columns. There are 14 features, in addition to one paining ID column and one target label column. The classes (paintings): "The Persistence of Memory", "The Starry Night", and "The Water Lily Pond" are perfectly balanced at 562 observations each (33.3%), so accuracy is a valid metric and no resampling is needed. Features include: 3 numerical (emotion intensity 1–10, colours count, objects count); 4 ordinal Likert scores (sombre, content, calm, uneasy, stored as strings like "4 - Agree"); 1 free-text price field that will be turned in a numerical feature; 3 multi-select categoricals (room, who, season—comma-separated option lists); and 3 free-text responses (feeling description, food analogy, soundtrack description).

Some of the survey questions ask respondents to rate their level of agreement with a statement on a scale of 1-5. We call these features the Likert features as they follow the Likert rating scale. We find that the Likert features are extremely discriminative (Figure 1): "Water Lily Pond" scores highest on calm (4.47) and content (4.41) but lowest on uneasy (1.36), while "Persistence of Memory" shows the opposite pattern (sombre 3.91, uneasy 3.85). "Starry Night" falls between the two. The season feature also varies strongly by class with "Water Lily Pond" associating with spring/summer while "Persistence of Memory" associates mostly with fall (Figure 2). Text features show class-specific vocabulary (e.g., "calm", "peaceful" for Water Lily; "time", "dread" for Persistence), with feeling descriptions averaging 13.7 words and food analogies only 3.4 words. Numerical features exhibit strong right skew in colours and objects counts (skewness > 11, values up to 100).

2.2 Data Issues and Preprocessing

Missing values affect all 14 feature columns (3.9–5.0% per column). However, we find that the rate of missing values is highly student specific: 93.2% of students have no missing values, while 24 students have all three rows affected. Rather than removing responses with missing values, we impute with per-class medians for numerical/Likert features and empty strings for text. Additionally, we find large outliers in features such as colours

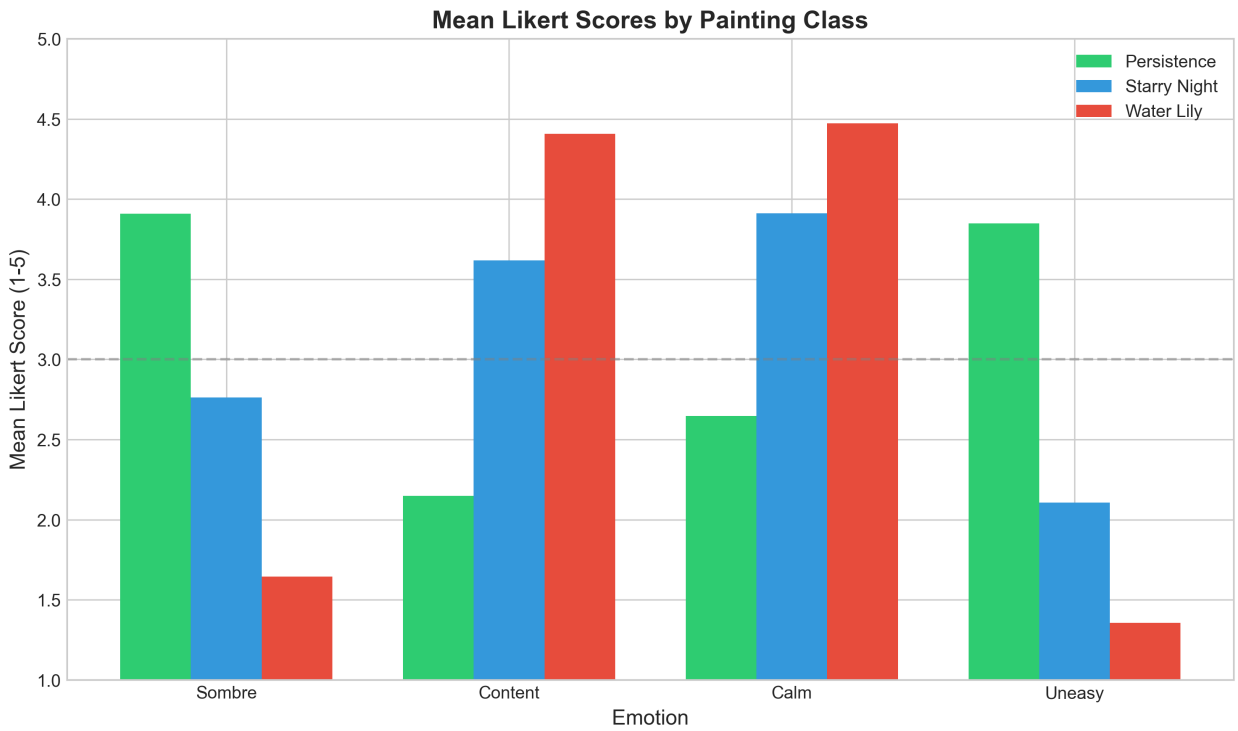

Figure 1: Mean Likert scores by painting class. The four emotions clearly separate the three classes.

count and objects count, where respondentts where asked how many objects and colours are in the painting. To sanitize these features we clip entries at the 95th percentile and replace with the max/min allowed value. The price column requires regex extraction from inconsistent free text (”$5”, ”300 dollars”, ”100000000”); after parsing 99.2% of entries, we clip at the 99th percentile and apply $\log(1 + x)$ to handle the extreme skew, since prices range from single digits to millions of dollars. 306 feeling descriptions are under 3 words but are retained as potential signal. Some free text responses contain words of objects found in the paintings (e.g. ”clocks”, ”lily”). We sanitize these responses by removing these words as survey instruc- tions stated that no such words should be used in responses.

Preprocessing pipeline: Likert columns are converted to numeric values (1–5) with no further scaling, as they are already on a consistent scale. Numerical features (emotion, colours, objects) and the cleaned price are z-score normalized to ensure comparable magnitudes. Multi-select fields are multi-hot encoded into $\sim$14 binary columns. Finally, we will try two different text-vectorization schemes. We will evaluate these methods based on model performance and memory usage as they must adhere to the 10MB file limit imposed by the project rules.

The first text-vectorization method is to use TF-IDF. TF-IDF vectorizes text by measuring a word’s importance to a response relative to its frequency across all responses. This down-weights common words like ”and” or ”the” while increasing the importance of

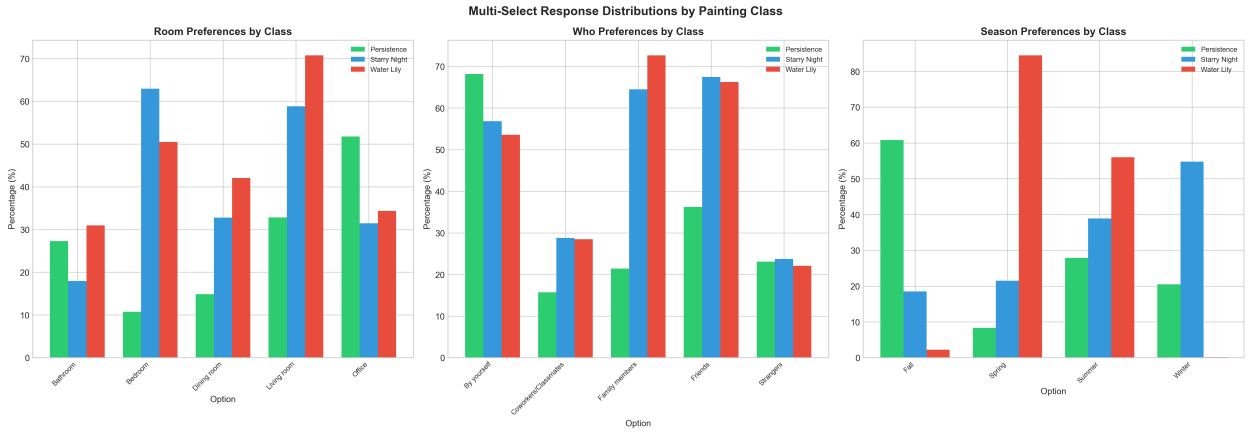

Figure 2: Multi-select response distributions by class. Season preferences show the strongest differentiation.

rarer, specific words.

The second method we will use is to create a meaningful vector embedding from all the words in a response. To convert a multi-word response to a vector we will use a hashing vectorizer. We first define a max vocabulary size D, and then each word in a response is hashed and the result is calculated mod D. The resulting value is taken as the index to one-hot encode the $D \times 1$ text-vector. This hashing-indexing method is applied to each word in the response to create a $D \times 1$ vector input. Then we will train a Neural Network to create text-embeddings from these one-hot encoded vectors, where the target output is a text-embedding from a pre-trained open-source model. This approach is more time-consuming than TF-IDF so we may implement it if time permits.

2.3 Data Splitting and Leakage Prevention

Each student wrote 3 responses, one per painting, creating dependence within each student's rows. If split across sets, the model could learn student-specific style rather than painting features, constituting data leakage. We intend to use the `GroupShuffleSplit` method from `scikit-learn` with `unique_id` as the grouping key, splitting into train (337 students, 1,011 rows, 60%), validation (112 students, 336 rows, 20%), and test (113 students, 339 rows, 20%).

2.4 Key Insights and Model Selection

The strong linear separability of the Likert features motivates logistic regression as a baseline, with regularization to handle the correlated Likert pairs. The mix of numerical, ordinal, binary, and sparse features motivates a use of Random Forest Tree, which can handle the heterogeneous input types without requiring extensive preprocessing and is robust to multicollinearity. The class-specific vocabulary in text features and nonlinear interactions between feature types also motivates a use of neural networks, whose hidden layers can learn richer representations across the combined feature space.

3 Methodology

Our group decided to select Logistic Regression, Random Forests, and Neural Networks as our three models for prediction. We believe that the Logistic Regression model will set a baseline for our final accuracy, as it is essentially a simplified version of a Neural Network. However, we will tune each model equally in order to ensure identical treatment of each model. The motivation for each model's selection was given in Section 2.4.

We will train the Random Forests using `scikit-learn`. Random forests do not require an optimizer (such as Adam or SGD) but we later specify the chosen hyperparameters that we tune.

We will train the Logistic Regression and Neural Network models using PyTorch. For these models, we will use the Adam optimizer to perform gradient descent. The Adam optimizer is a modified version of SGD which uses "momentum" (an average of past gradients) to update weights, and it is known to give more stable convergence than SGD.

The hyperparameters for our optimization process are detailed in Table 1. We vary the base learning rate, which determines the step size taken during gradient descent, and try multiple learning rate schedulers, which adjust this rate during training to improve convergence. To stabilize training, we tune the warmup %, defined as the proportion of total steps where the learning rate gradually increases to the base value. This allows the model to explore the loss landscape and find a better local minimum. Finally, we implement early stopping to prevent the model from overfitting to the training data. This is controlled by patience, the number of steps to wait for improvement; min delta, the minimum change in the monitor metric required to qualify as an improvement; and the metric itself (e.g., validation loss).

Category	**Hyperparameter**	**Common Search Values / Options**
Learning Rate	Base Learning Rate (η)	$10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}$
LR Schedulers	Type	Constant, Cosine Annealing, OneCycleLR
	Warmup %	5%, 10%, 30% (percentage of total training steps)
Early Stopping	Patience	$10, 20$ (epochs to wait before stopping)
	Min Delta	$10^{-2}, 10^{-3}$ (minimum metric change to qualify as improvement)
	Monitor Metric	Validation Loss, Validation Accuracy, Validation F1

Table 1: Hyperparameter Search Space for Optimizer Tuning

For the Random Forest we tune the forest size and the tree depth, which dictate the number of individual decision trees and their maximum depth respectively. For the tree depth we tune different metrics such as setting a max depth and setting the minimum number

of samples needed to split. We also tune "max features" which sets how many features are considered at each splitting point. Finally we also optimize the data sampling methods and the criterion used to decide optimal splits. These hyperparameters are given in Table 2.

Category	Hyperparameter	Common Search Values / Options
Forest Size	n_estimators	100, 200, 500, 1000
Tree Depth	max_depth	None (unlimited), 10, 20, 30, 50
	min_samples_split	2, 5, 10
	min_samples_leaf	1, 2, 4
Feature Subset	max_features	'sqrt' (default), 'log2', None (all), 0.3, 0.5
Data Sampling	bootstrap	True, False
	max_samples	None, 0.5, 0.7, 0.8 (fraction of training set)
Impurity	criterion	Gini Impurity, Entropy (Log Loss)

Table 2: Hyperparameter Search Space for Random Forest Evaluation

Finally, for Logistic Regression we give the hyperparameter choices in Table 3. For this model we only tune the type of regularization and its strength. The regularizer penalizes the model for large weight magnitudes, encouraging equal contribution from all features for predictions which helps to avoid overfitting. The strength of the regularization is also tuned in order to achieve good balance between avoiding overfitting and achieving high training accuracy.

Category	Hyperparameter	Common Search Values / Options
Regularization	Penalty Type	L1 (Lasso), L2 (Ridge), None
	Strength (C)	$10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100$ (Inverse of λ)

Table 3: Hyperparameter Search Space for Logistic Regression Evaluation

We tune several hyperparameters to optimize the Multi-Layer Perceptron (MLP) performance, as given in Table 4. For the architecture, we vary the number of hidden layers and the number of neurons per layer to find the appropriate model capacity for the task's complexity. We evaluate different activation functions, which determine the non-linear transformation of the input signal at each neuron. Additionally, we toggle skip connections, which are residual shortcuts that allow gradients to flow more easily through deeper architectures. To ensure the model generalizes well, we tune two primary regularization parameters: the dropout rate, which randomly deactivates a percentage of neurons during training to prevent over-reliance on a given neuron, and weight decay which adds a penalty to the loss function based on the magnitude of the weights to discourage overfitting.

Category	Hyperparameter	Search Space / Options
Architecture	Hidden Layers	$1, 2, 3, 4$
	Neurons per Layer	$32, 64, 128, 256, 512$
	Activation Function	ReLU, GELU
	Skip Connections	Yes, No
Regularization	Dropout Rate	$0.0, 0.1, 0.2, 0.3, 0.5$
	Weight Decay (L_2)	$10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}$

Table 4: MLP Architecture and Regularization Hyperparameters

For evaluation metrics, we will use the accuracy, precision, recall, and F1 scores. Accuracy is measured as by Accuracy $= \frac{\text{Correct Guesses}}{\text{Total Guesses}}$. We chose accuracy because it is an easily interpretable metric which suits our balanced dataset. We also plan to use the F1, recall and precision scores given by $F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$, where Recall $= \frac{TP}{TP+FN}$ and Precision $= \frac{TP}{TP+FP}$. These score will help us identify if our models struggle with identifying specific classes out of the three paintings. This will help us interpret our model performance and inform decisions on training.

4 Contributions and Learning

We keep the contributions section brief for the project proposal. Amaan and Kabir worked on the data exploration section while Alexandros and Adam worked on the methodologies section.